You're right. At JetBot speed, even if there is measurable drivetrain lag, it is probably **not decision-significant**. A "crossing obstacle" demo would mostly be solved by a simple time-to-collision rule:

> "If I will collide, brake; otherwise continue."

That would not be a convincing reason to use MPPI.

For your robot as it exists, I would not claim MPPI is necessary. Its dynamics are close to **kinematic**: it can more or less go where you point it, and a good path tracker is likely enough.

The cleanest way to add a genuinely predictive element without needing speed is to make the robot manage something whose state is **not directly controllable and depends on prior motion**.

Best option: add a small trailer and make it reverse through a tight gate

Have the JetBot tow a little two-wheel trailer or cart, tracked by Vicon. Then set up a known maneuver where it must back the trailer into a parking bay or through a narrow S-shaped gate.

Now the robot cannot simply say:

> "I am left of the path; steer right."

A right command may initially move the trailer the wrong way, and by the time the trailer angle is visibly bad, it may be too late to correct without pulling forward and trying again.

That is a real low-speed predictive-control problem:

- robot pose and trailer pose both matter;
- the trailer has a swept collision envelope;
- reverse motion is nonlinear and locally unstable;
- present steering choices determine the trailer's orientation several seconds later;
- obstacle clearance is nonconvex.

MPPI can roll out hundreds of short control sequences and ask:

> "Which left/right wheel-command sequence gets both the JetBot and the trailer through the gate without clipping a wall?"

No high inertia or racing speed required.

A conventional nonlinear MPC, DDP, or a carefully designed trailer controller could also solve it. The point is not that only MPPI can do it; the point is that **a future-looking optimizer becomes meaningfully justified**, whereas plain cross-track feedback no longer is.

A second, more visibly "dynamic" option: a suspended payload

Mount a light pendulum or hanging payload above the JetBot and track both the robot and the pendulum bob with Vicon.

The task becomes:

> Reach the goal quickly, remain in the corridor, and keep the payload swing below a limit.

A simple tracker makes a sharp turn, sees the pendulum swing afterward, then tries to correct—after it has already injected energy into the load. MPPI can predict that a sharp turn now will create swing half a second later, so it begins turning earlier, reduces speed, or countersteers.

You would need to make the pendulum deliberately responsive enough to matter: a longer, low-damping pendulum and a fairly tight S-turn work better than a short stiff one.

My honest recommendation

For the strongest JetBot project, I would choose:

> **Vicon-tracked JetBot towing a trailer, performing reverse docking through a narrow gate.**

It gives you a compelling before-and-after comparison:

- **Pure Pursuit / PID:** tracks the robot's center path but jackknifes or clips the trailer.
- **MPPI:** predicts the articulated system's future geometry and chooses a corrective maneuver before the trailer is visibly in trouble.

That would demonstrate a real advantage of predictive local control without pretending your JetBot is a race car.